

**Project Development Phase
Performance Test**

Date	11 September 2026
Team ID	NA
Project Name	Metro Ticket Generating System in ServiceNow
Maximum Marks	

Model Performance Testing:

S.No.	Parameter	Screenshot / Values
1.	Data Rendered	• Data Source: ServiceNow Enterprise Instance (Personal Developer Instance) • Primary Tables: sc_request, sc_req_item, u_metro_station_details • Metrics Rendered: Total daily tickets generated, route distribution frequencies, single vs. return journey splits, fare collection aggregates, and QR scan audit logs.
2.	Data Preprocessing	• Null Value Treatment: Handled unassigned passenger counts by enforcing a default floor of 1. • String Standardization: Trimmed whitespace and standardized station naming conventions from u_metro_station_details. • Type Casting: Converted ticket amounts and journey flags (type_of_journey) from string objects to numeric integers/decimals for accurate aggregation. • Timestamp Normalization: Formatted creation dates into standard UTC and local transit operating hours.
3.	Utilization of Filters	• Station Slicers: Interactive filters allowing operators to isolate specific origin and destination nodes (e.g., Ameerpet, Madhapur, LB Nagar, Kukatpally). • Journey Type Slicer: Toggle between Single Journey (1) and Return Journey (2). • Date Range Slicer: Dynamic time filtering for peak-hour analysis (Morning 08:00–11:00 vs. Evening 17:00–20:00). • Payment Mode Filter: Segment data by UPI, Credit/Debit Card, Cash, or Others.
4.	Calculation fields Used	• Total Revenue Generated: $\text{Total_Revenue} = \text{SUMX}(\text{sc_req_item}, \text{sc_req_item.price})$

		• Total Tickets Sold: Total_Tickets = DISTINCTCOUNT(sc_req_item.sys_id) • Average Fare per Trip: Avg_Fare = AVERAGE(sc_req_item.price) • Peak Hour Volume Index: Peak_Volume = CALCULATE(COUNT(sc_request.sys_id), HOUR(sc_request.sys_created_on) >= 8 && HOUR(sc_request.sys_created_on) <= 11)
5.	Dashboard design	No of Visualizations / Graphs
6.	Story Design	No of Visualizations / Graphs